

SocialApp — Build Week 3

Architettura, storia ed esempi di codice del progetto e di tutto il lavoro svolto

Aggiornato il 21 luglio 2026 · React 19 + Redux Toolkit + Socket.IO + JSON Server

1. Cos'è il progetto

SocialApp è un social network didattico (esercizio "Build Week 3", corso Epicode): autenticazione, feed con post/commenti/like, profilo utente, messaggistica privata in tempo reale, notifiche, ricerca e ora anche un sistema di follower. Il frontend è React 19 + Redux Toolkit; il backend è un mock JSON Server + JSON Server Auth (autenticazione JWT, permessi in stile Unix `rwX` per collection), esteso con un livello Express custom per gestire upload di grandi dimensioni e comunicazione in tempo reale via Socket.IO.

2. Come è costruito il progetto

Struttura delle cartelle

server/	
db.json	dati demo + stato runtime (letto dal server)
db.seed.json	stesso schema, versione "modello" per il repo
routes.json	permessi json-server-auth per collection
server.js	bootstrap Express + Socket.IO (unico entry point)
realtime.js	eventi realtime + notifiche (hook su router.render)
src/	
main.jsx	bootstrap tecnico: crea il root DOM, monta <App />
App.jsx	radice logica: Redux Provider, BrowserRouter, ciclo di vita socket/notifiche/badge non letti
socket.js	client Socket.IO singleton
app/	
store.js	configureStore: un reducer per feature + middleware
toastMiddleware.js, loadingBarMiddleware.js	
api/	un file per risorsa, solo chiamate HTTP (axios)
features/	uno slice Redux per dominio (auth, posts, comments, messages, users, notifications, search, follow, presence) – createAsyncThunk + extraReducers
components/	componenti "di presentazione", divisi per dominio (posts/, comments/, messages/, profile/, common/, ...)
pages/	componenti "contenitore": leggono lo store, orchestrano i componenti di presentazione
hooks/	hook riusabili (useDebounce, usePresenceSocket, ...)
routes/	AppRouter, ProtectedRoute, PublicRoute
utils/	funzioni pure: validazioni, date, JWT, toast, confirm

Pattern service + slice (ripetuto identico per ogni dominio)

Ogni risorsa ha un file in `src/api/` che parla solo HTTP (nessuna logica Redux), e uno slice in `src/features/` che lo usa dentro `createAsyncThunk`. Esempio reale, `src/api/postsService.js`:

```
export async function fetchPosts({ page = 1, limit = 5, q, userIds } = {}) {
  const params = new URLSearchParams()
  params.set('_sort', 'createdAt')
  params.set('_order', 'desc')
  params.set('_page', page)
  params.set('_limit', limit)
  if (q) params.set('q', q)
  if (userIds) userIds.forEach((id) => params.append('userId', id))
  const { data, headers } = await httpClient.get(`/posts?${params.toString()}`)
  return { posts: data, totalCount: Number(headers['x-total-count'] ?? data.length) }
}
```

Nello slice, ogni azione asincrona segue sempre la stessa forma — `pending` imposta lo stato di caricamento, `fulfilled` aggiorna i dati, `rejected` salva l'errore — così un nuovo collaboratore che ha capito uno slice li ha capiti tutti:

```
export const fetchPosts = createAsyncThunk('posts/fetchPosts', async ({ page, limit }, { rejectWithValue }) => {
  try {
    const { posts, totalCount } = await postsService.fetchPosts({ page, limit })
    return { posts, totalCount, page }
  } catch (err) {
    return rejectWithValue(err.message)
  }
})
// ...
builder
  .addCase(fetchPosts.pending, (state) => { state.status = 'loading' })
  .addCase(fetchPosts.fulfilled, (state, action) => {
    state.status = 'succeeded'
    state.items = action.payload.page === 1 ? action.payload.posts : [...state.items, ...action.payload.posts]
  })
  .addCase(fetchPosts.rejected, (state, action) => { state.status = 'failed'; state.error = action.payload })
```

Autenticazione: JWT in localStorage + interceptor Axios

Al login, il token ricevuto da json-server-auth viene salvato in `localStorage`. Un interceptor Axios lo allega automaticamente a *ogni* richiesta, così i service in `src/api/` non devono mai occuparsene esplicitamente; un secondo interceptor intercetta le risposte `401` ed emette un evento globale che `App.jsx` ascolta per fare logout automatico (`src/api/httpClient.js`):

```
httpClient.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_STORAGE_KEY)
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

httpClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      window.dispatchEvent(new CustomEvent('auth:expired'))
    }
    return Promise.reject(new Error(/* ... */))
  },
  )
```

Questo stesso pattern "evento globale disaccoppiato" (`CustomEvent` + `window.addEventListener`) è stato riusato più volte in questa sessione per toast e modale di conferma (vedi passaggio 10) invece di inventare ogni volta un meccanismo diverso.

Permessi del backend mock

`server/routes.json` mappa ogni collection a un codice ottale in stile Unix (`rwX` per proprietario/loggati/pubblico) interpretato da json-server-auth:

```
{
  "users": 640, "posts": 640, "comments": 640, "likes": 640,
  "messages": 660, "conversations": 660,
  "notifications": 640, "follows": 640
}
```

`640` = solo il proprietario (campo `userId` sulla risorsa) può scrivere/eliminare, chiunque sia loggato può leggere. `messages` / `conversations` sono a `660` (chiunque loggato scrive) perché json-server-auth riconosce la proprietà solo su un singolo campo `userId`, e queste risorse hanno invece *due* partecipanti — l'autorizzazione fine è quindi applicata lato client (vedi passaggio "fasi precedenti" più sotto).

3. Flusso dell'applicazione

Il client comunica con il server mock su due canali paralleli, entrambi sullo stesso processo/porta:



In pratica: quando un utente invia un messaggio, commenta, mette "mi piace" o inizia a seguire un altro utente, il router di json-server esegue la scrittura reale su `db.json` come sempre; subito dopo, `realtime.js` intercetta quella risposta (senza duplicare la logica di validazione/autorizzazione) ed emette l'evento via Socket.IO solo agli utenti coinvolti (stanza `user:<id>`), oltre a creare — se pertinente — una notifica persistita nel database. Lo stesso canale WebSocket porta anche gli eventi di presenza (`presence:online` / `presence:offline`), calcolati da un contatore di connessioni per utente mantenuto in memoria dal server.

server/realtime.js — un ramo dell'hook router.render (schema ripetuto per ogni evento)

```
router.render = (req, res) => {
  const { method, path } = req
  const data = res.locals.data // la risorsa appena scritta da json-server

  if (method === 'POST' && path === '/messages' && res.statusCode === 201) {
    const conversation = server.db.get('conversations').find({ id: data.conversationId }).value()
    io.to(`user:${conversation.participant1Id}`).emit('message:new', data)
    io.to(`user:${conversation.participant2Id}`).emit('message:new', data)
    const recipientId = conversation.participant1Id === data.userId
      ? conversation.participant2Id : conversation.participant1Id
    createNotification(server, io, { userId: recipientId, actorId: data.userId,
      type: 'message', message: `${actorName(server, data.userId)} ti ha inviato un messaggio` })
    return defaultRender(req, res) // risposta HTTP invariata, come faceva json-server di default
  }
  // ... altri rami per PATCH/DELETE messaggi, POST comments/likes/follows/conversations
  return defaultRender(req, res)
}
```

4. Le fasi precedenti a questa sessione

Questa sessione non è partita da zero: il progetto esisteva già, con una sua storia documentata in `CHECKPOINT.md`. Riassunto di come si è arrivati al punto di partenza:

Fase 2 (milestone M0-M9) — l'implementazione di base

Scaffold React + Vite adattato (il progetto era già stato inizializzato con React 19/Vite 8/ oxcint prima di iniziare, invece di React 18/ESLint come originariamente ipotizzato in fase di design — si è scelto di adattarsi allo scaffold esistente). Backend mock JSON Server + JSON Server Auth con dati demo, store Redux con uno slice per dominio (auth/posts/comments/users/ messages), autenticazione completa, layout e navbar, profilo utente, feed con CRUD di post/ commenti/like, messaggistica privata con CRUD completo, stati di caricamento/errore/vuoto su ogni schermata, autorizzazioni lato client. Verificato con un test end-to-end reale in browser (Playwright headless).

Due decisioni di questa fase restano visibili in tutto il codice attuale: la collection `likes` separata (non un array dentro il post) perché json-server-auth rifiuta la scrittura di un campo su una risorsa di cui non si è proprietari — un "mi piace" di un altro utente modificherebbe il post altrui e verrebbe respinto con 403; e `App.jsx` come radice logica dell'app (Redux `Provider` + `BrowserRouter` vivono lì), con `main.jsx` ridotto a puro bootstrap tecnico (crea il nodo DOM, monta `<App />`) — una separazione voluta esplicitamente, mai spostata nemmeno con l'espansione di questa sessione.

Restyling grafico ispirato a LinkedIn ("inClone")

Rebranding di navbar e pagine di autenticazione, layout del feed a tre colonne (mini-profilo sticky + feed + card "Novità"), avatar di fallback con gradiente deterministico (basato sull'id utente) al posto di un cerchio blu piatto, banner di copertina sul profilo. Aggiunti campi minori: `jobTitle` (titolo professionale opzionale), split Nome/Cognome nel form di registrazione (uniti in `fullName` prima dell'invio), supporto immagine nei post (il campo `imageUrl` esisteva già nello schema ma non era mai esposto in UI).

Upload reale per l'avatar profilo e i media dei post

`ProfileEditForm` ha sostituito il campo URL testuale con un `<input type="file">`: il file scelto viene letto con `FileReader.readAsDataURL` e la stringa risultante (base64) salvata direttamente come `avatarUrl` (limite 2MB lato client). Stessa tecnica poi estesa al campo immagine/video dei post (limiti 3MB foto / 50MB video). Questa scelta — niente object storage esterno, tutto dentro `db.json` — ha avuto una conseguenza tecnica diretta sulla fase successiva.

Bootstrap Express custom per superare il limite di json-server

Un video da 50MB codificato in base64 pesa circa 68MB, ma sia `json-server` che `json-server-auth` hanno un proprio body-parser con limite **hardcoded a 10MB**, non esposto da nessun flag CLI. Soluzione: sostituire l'avvio via CLI con un bootstrap Express scritto a mano (questo è già `server/server.js`, poi ulteriormente esteso in questa sessione per Socket.IO):

```
server.use(jsonServer.defaults({ bodyParser: false })) // disabilita il parser da 10MB
server.use(express.json({ limit: BODY_LIMIT })) // BODY_LIMIT = '70mb'
server.use(express.urlencoded({ extended: false, limit: BODY_LIMIT }))
server.use(auth.rewriter(routes))
server.use(auth) // trova il body già' parsato (req._body === true) e salta il proprio parser da 10MB
server.use(router)
```

Verificato all'epoca con richieste PATCH reali fino a 30MB di body, e con un test di guardia (owner diverso → 403 confermato invariato) per assicurarsi che il bypass del parser non avesse anche bypassato l'autorizzazione.

5. Passaggi fatti in questa sessione

1 Lettura del checkpoint e risoluzione di un merge in sospeso

Letto `CHECKPOINT.md` per il contesto del progetto (stack, decisioni prese, limiti noti). Un `git status` ha rivelato un merge Git non concluso (`All conflicts fixed but you are still merging`) con due file nuovi non ancora in staging: `server/db.seed.json` e `src/hooks/useInterval.js` . Il messaggio di merge pendente (`.git/MERGE_MSG`) indicava `server/db.seed.json` come unico file in conflitto.

Un controllo con `git grep` per marcatori `<<<<<<` non aveva dato risultati — ma il file era ancora *untracked*, e `git grep` di default ignora i file non tracciati. Aggiungendolo esplicitamente e ricontrollando è emerso che conteneva ancora marcatori di conflitto reali (`<<<<<< HEAD:server/db.json ... ===== ... >>>>>> origin/main:server/db.seed.json`) nel mezzo della collection `messages` : due messaggi reali della stessa conversazione con lo stesso `id` duplicato. Risolti a mano conservando entrambi i messaggi e riassegnando id univoci, poi il merge è stato concluso con un commit (`Merge remote-tracking branch 'origin/main' into Lucho`).

In una verifica successiva sono stati trovati anche altri id duplicati preesistenti (due utenti diversi con `id: 5` , due like con lo stesso `id`) in `db.seed.json` — segnalati all'utente ma non corretti in quel file, perché è un residuo del merge non letto a runtime dal server (che legge invece `server/db.json` , risultato pulito).

2 Analisi architetturale approfondita e piano approvato

Per la richiesta di implementare 4 funzionalità grandi (messaggistica realtime, notifiche, ricerca+paginazione, modifica commenti), è stata usata la modalità di pianificazione: un agente di esplorazione dedicato ha mappato l'intera architettura (pattern `createAsyncThunk` + `extraReducers` nelle slice Redux esistenti, pattern dei service HTTP in `src/api/` , struttura di `server/server.js` , schema del database, permessi `json-server-auth` in `routes.json`), poi sono stati letti direttamente i file critici e ispezionati i sorgenti di `json-server` / `json-server-auth` in `node_modules` per verificare due dettagli tecnici prima di scrivere codice: il secret JWT usato per firmare i token (hardcoded in `json-server-auth/dist/constants.js` , riusabile per autenticare i socket) e il meccanismo ufficiale `router.render` che json-server espone per intercettare le risposte.

Sono state poste esplicitamente all'utente due domande architetturali prima di procedere (tramite scelta multipla): se introdurre TypeScript (risposta: no, restare in JavaScript puro, coerente con "mantieni la stessa architettura") e se usare `socket.io` o `ws` grezzo (risposta: `socket.io` , per riconnessione automatica e gestione errori integrate). Il piano dettagliato risultante è stato scritto su file ed esplicitamente approvato dall'utente prima di toccare qualunque codice.

3 Backend — Messaggistica realtime e notifiche

- `server/server.js` : la creazione dell' `http.Server` (prima implicita dentro `server.listen()`) è stata resa esplicita e spostata *prima* della registrazione dei middleware, così `socket.io` può agganciarsi alla stessa istanza/porta invece di aprire un secondo processo.
- Nuovo `server/realtime.js` con due responsabilità: `authenticateSocket(io)` verifica il JWT nell'handshake (`socket.handshake.auth.token`) con lo stesso secret di `json-server-auth` ed estrae l'id utente dal claim `sub` ; `attachRealtime(server, router, io)` sovrascrive `router.render` per intercettare ogni risposta subito dopo la scrittura reale su `db.json` ed emettere l'evento giusto: `message:new` / `message:updated` / `message:deleted` , `conversation:new` , oltre a creare notifiche su nuovo commento/like/messaggio.
- Un piccolo middleware Express aggiuntivo cattura la risorsa *prima* di un `DELETE /messages/:id` (la cui risposta di default è `{}`), perché serve sapere a quale `conversationId` apparteneva il messaggio eliminato per notificare i partecipanti giusti.
- Nuova collection `notifications` (`id, userId, actorId, type, message, postId?, conversationId?, read, createdAt`) con permesso `640` : qui `userId` rappresenta correttamente il destinatario, quindi il modello di ownership di json-server-auth funziona senza bisogno del workaround a `660` usato per messaggi/conversazioni.

Bug reale trovato e risolto: crash cascade-delete di json-server

Durante un test end-to-end reale, un `DELETE /comments/:id` rispondeva `500`. Lo stack trace ha portato a `node_modules/json-server/lib/server/mixins.js`: prima di ogni cancellazione, json-server scansiona *ogni* campo che termina in "Id" in *tutte* le collection per rimuovere le risorse orfane collegate, e va in crash (`TypeError: Cannot read properties of null`) se incontra un valore `null`, perché `lodash-id.getById` non lo gestisce. Le notifiche appena introdotte creavano campi come `postId: null` quando non applicabile (es. una notifica di messaggio non ha un post associato), attivando il bug. Risolto omettendo del tutto la chiave invece di impostarla a `null` quando non pertinente.

Bug reale trovato e risolto: condizione di corsa sull'invio messaggi

Il server emette `message:new` a *entrambi* i partecipanti della conversazione, mittente incluso (per semplicità di sincronizzazione). Se quell'evento socket arrivava al mittente *prima* della risposta HTTP della sua stessa richiesta `POST /messages`, il messaggio veniva aggiunto due volte allo stato Redux (una dal thunk REST, una dall'evento socket) e il contatore totale usato per la paginazione veniva incrementato due volte. Risolto aggiungendo un controllo "esiste già questo id nello stato?" in entrambi i reducer coinvolti (`sendMessage.fulfilled` e `messageReceived`), non solo in uno dei due.

4 Frontend — Le 4 funzionalità

Messaggistica in tempo reale

Nuovo `src/socket.js`: istanza singleton di `socket.io-client` con `autoConnect: false`, connessa/disconnessa da `App.jsx` in base allo stato di login (stesso punto in cui già si gestiva l'evento `auth:expired`). Nuovo hook `useConversationSocket`, montato una sola volta in `MessagesPage` (padre di lista conversazioni e vista conversazione), che sottoscrive gli eventi e li traduce in azioni Redux tramite nuovi reducer "puri" aggiunti a `messagesSlice` (prima aveva solo `extraReducers` per i thunk REST): `messageReceived`, `messageUpdatedFromSocket`, `messageDeletedFromSocket`, `conversationReceived`. Rimosso `src/hooks/useInterval.js`: un hook di polling generico completo e funzionante, ma mai importato da nessuna parte del codice — quindi "sostituire il polling" si è tradotto in eliminare codice morto, non in disattivare un meccanismo realmente attivo.

Notifiche in tempo reale

Nuovo slice `notifications` (thunk `fetchNotifications` / `markNotificationRead` + reducer `notificationReceived` per l'evento socket) e componente `NotificationBell` in navbar: icona campana con badge del conteggio non letti, dropdown con l'elenco, click che segna come letta e naviga al contenuto pertinente.

Ricerca e paginazione

Nuova pagina `/search` con due tab (Utenti/Post), basata sulla ricerca full-text già integrata in json-server (parametro `?q=`) invece di costruire un motore di ricerca dedicato. Nuovo slice `search` indipendente da `posts` / `users` per non introdurre regressioni nelle slice esistenti. Paginazione aggiunta anche a commenti e messaggi: per i messaggi l'implementazione è diversa da post/commenti perché una chat mostra prima i messaggi *più recenti* — il servizio interroga `_order=desc` (pagina 1 = ultimi N), inverte l'ordine lato client, e "carica messaggi precedenti" *antepone* invece di accodare, ripristinando manualmente l'offset di scroll (catturato prima del fetch) per evitare il salto visivo tipico di questo pattern.

Modifica dei commenti

Prima i commenti si potevano solo creare ed eliminare. È stata scoperta durante l'analisi che la modifica dei *messaggi* (`editMessage` + interfaccia inline in `MessageBubble`) era già completamente implementata: la modifica dei commenti è

stata quindi realizzata come replica 1:1 di quel pattern già collaudato (stesso PATCH lato servizio, stesso thunk, stessa interfaccia di modifica inline in `CommentItem`), invece di inventarne uno nuovo.

5 Verifica end-to-end

Non essendo disponibile uno strumento di automazione browser in questo ambiente, la verifica è stata fatta con script Node dedicati (non simulazioni: chiamate HTTP e socket reali contro un'istanza del backend isolata su una porta di test), poi ripuliti a fine test:

- login di due utenti reali, connessione socket autenticata per entrambi;
- un tentativo di connessione socket con token non valido, verificato come effettivamente rifiutato;
- invio, modifica (contenuto e stato letto/non letto) ed eliminazione di un messaggio, con verifica puntuale che l'*altro* utente riceva l'evento socket corrispondente;
- creazione di un commento e di un "mi piace" su un post altrui, con verifica che l'autore del post riceva `notification:new` del tipo corretto, che la notifica sia persistita e recuperabile via REST, e che il "segna come letta" funzioni;
- ricerca utenti/post con parametri parziali, paginazione di commenti e messaggi con `_page` / `_limit`;
- modifica di un commento proprio, e verifica che lo stesso commento *non* sia modificabile da un altro utente via chiamata API diretta (risposta attesa e ottenuta: `403`).

`npm run lint` e `npm run build` eseguiti puliti dopo ogni blocco di modifiche, non solo alla fine.

⚠ Limite dichiarato all'utente

Non è stato possibile testare l'interfaccia in un vero browser (click, rendering, scroll) per mancanza di uno strumento di automazione browser in questo ambiente — è stato segnalato esplicitamente invece di dichiarare il lavoro "verificato al 100%".

6 Documentazione aggiornata

`CHECKPOINT.md` aggiornato con le nuove decisioni tecniche e la motivazione dietro ciascuna (perché `socket.io` e non `ws`, perché il permesso `640` basta per `notifications` a differenza di `messages` / `conversations`, perché la paginazione dei messaggi è invertita rispetto a post/commenti), i limiti noti rivisti (rimossi quelli non più veri, es. "nessun WebSocket") e una voce di changelog datata in fondo al file. `README.md` aggiornato: tabella endpoint con i nuovi parametri di query, nuova sezione dedicata al WebSocket, struttura cartelle aggiornata con i file nuovi, funzionalità completate e limiti noti — come richiesto esplicitamente dalle istruzioni del repository (`CLAUDE.md`), che impongono di aggiornare il changelog dopo modifiche significative senza dover chiedere conferma.

7 Diagnosi "il sito non funziona"

Il backend che l'utente aveva già in esecuzione (porta 3001, avviato prima di questa sessione di modifiche) era rimasto attivo con il codice *vecchio*: Node non ricarica automaticamente un processo server quando i file cambiano (a differenza del frontend Vite, che fa hot-reload). Verificato in modo puntuale interrogando `/socket.io/?EIO=4&transport=polling` sulla porta 3001: rispondeva `404`, prova diretta che quel processo non aveva `socket.io` agganciato. Il processo è stato terminato e `npm run dev:all` è stato rilanciato; dopo il riavvio sono stati confermati uno per uno: avvio pulito nei log, tutti i moduli nuovi (`socket.js`, slice nuove, componenti nuovi) trasformati correttamente da Vite senza errori 500, lo stesso endpoint `/socket.io/` che ora risponde `200`, e un login reale riuscito sul backend appena riavviato.

Successivamente, un login segnalato come fallito per `SaltyGhosty@gmail.com` è risultato *non* essere un bug dopo aver ispezionato `server/db.json`: l'utente con username `SaltyGhosty` risultava registrato con l'indirizzo email `skibidi@gmail.com` — il login richiede l'email reale con cui ci si è registrati, non lo username, e i due non coincidevano in questo caso.

8 Miglioramenti estetici (prima tornata)

- **Tema scuro**: toggle in navbar (icona sole/luna), basato sull'attributo `data-bs-theme` nativo di Bootstrap 5.3 (non una libreria esterna). Uno script inline in `index.html`, eseguito *prima* del mount di React, applica subito il tema salvato in `localStorage` (o la preferenza di sistema) così non c'è nessun flash del tema sbagliato al caricamento. Sono state corrette anche alcune utility Bootstrap non "theme-aware" per default (`text-dark`, `bg-light`, `bg-white`, usate in diversi punti dell'app per lo stile di default) con override CSS scoped a `[data-bs-theme='dark']`.
- **Toast di conferma**: nuovo sistema a evento globale (`CustomEvent`, stesso pattern già usato in precedenza per `auth:expired`) + un middleware Redux dedicato (`toastMiddleware.js`) con una mappa centralizzata "tipo di azione completata → messaggio", così aggiungere un nuovo toast in futuro significa aggiungere una riga sola, non modificare ogni componente.
- **Skeleton loader** animati (effetto shimmer via CSS) al posto dello spinner generico, nel feed, nei commenti, nei messaggi, nelle conversazioni e nella ricerca.
- **NotificationBell**: icona diversa per tipo di notifica (messaggio/commento/like) e piccola animazione di rotazione della campana quando il conteggio dei non letti aumenta (rilevato confrontando il valore precedente con `useRef`).
- **EmptyState**: icona racchiusa in un cerchio con sfondo attenuato invece dell'icona "nuda", più una dissolvenza in ingresso.
- **Micro-interazioni**: piccola animazione "pop" sul pulsante mi piace al click, transizioni morbide (150-200ms) su bottoni/card/link in tutta l'app invece di cambi di stato istantanei.

9 Nuove funzionalità: seguire utenti e stato online

Seguire un utente: nuova collection `follows` (`id`, `userId`, `followingId`, `createdAt`), progettata riutilizzando esattamente lo stesso schema già collaudato per `likes` (`userId` = chi compie l'azione), così il permesso `640` di `json-server-auth` funziona subito senza workaround: chiunque loggato può leggere (per contare follower/following), solo chi segue può creare/eliminare il proprio record. Nuovo slice `follow` con thunk `toggleFollow` che replica esattamente la logica già usata per `toggleLike` in `postsSlice` (cerca il record esistente nello stato, POST se assente/DELETE se presente). Nel backend, `realtime.js` genera anche qui una notifica in tempo reale ("X ha iniziato a seguirti") riutilizzando l'helper già scritto per commenti/like. In `ProfileCard`: bottone Segui/Segui già, contatori follower/seguiti.

Stato online: `realtime.js` mantiene una `Map` "userId → numero di connessioni attive" (non un semplice booleano): lo stesso utente può avere più schede/dispositivi aperti, e l'evento `presence:offline` deve essere emesso solo quando l'ultima connessione si chiude, non alla prima scheda chiusa. Alla connessione di un client, il server gli manda anche `presence:list` con tutti gli utenti già online in quel momento (altrimenti un client appena connesso non saprebbe chi era online prima del suo arrivo). Nuovo slice `presence` + hook `usePresenceSocket`, montato una sola volta in `App.jsx` per l'intera durata della sessione (non solo nella pagina messaggi), così il pallino verde pulsante compare ovunque sia rilevante: profilo, elenco conversazioni, risultati di ricerca. Il componente `Avatar` è stato esteso con un prop `online` opzionale che sovrappone il pallino via CSS, senza modificare la logica esistente per chi non lo usa.

src/features/follow/followSlice.js — stesso identico schema di toggleLike

```
export const toggleFollow = createAsyncThunk('follow/toggleFollow', async ({ followerId, followingId }, { getState }) => {
  const existing = getState().follow.items.find(
    (r) => r.userId === followerId && r.followingId === followingId,
  )
  if (existing) {
    await usersService.unfollowUser(existing.id)
    return { following: false, followId: existing.id }
  }
  const record = await usersService.followUser({ followerId, followingId })
  return { following: true, record }
})
```

server/realtime.js — contatore di connessioni per la presenza online

```
const onlineConnections = new Map() // userId -> numero di schede/dispositivi connessi

io.on('connection', (socket) => {
  const count = (onlineConnections.get(socket.userId) || 0) + 1
  onlineConnections.set(socket.userId, count)
  if (count === 1) io.emit('presence:online', socket.userId) // solo alla PRIMA connessione
  socket.emit('presence:list', [...onlineConnections.keys()]) // chi era già online

  socket.on('disconnect', () => {
    const remaining = (onlineConnections.get(socket.userId) || 1) - 1
    if (remaining <= 0) {
      onlineConnections.delete(socket.userId)
      io.emit('presence:offline', socket.userId) // solo quando si chiude l'ULTIMA scheda
    } else {
      onlineConnections.set(socket.userId, remaining)
    }
  })
})
```

10 Modal di conferma e ulteriori miglioramenti grafici

Modal di conferma: i `window.confirm()` nativi del browser (bruttini e non coerenti con lo stile dell'app) usati per eliminare un post e un messaggio sono stati sostituiti con un modale Bootstrap. Architettura identica al sistema toast: `src/utils/confirm.js` espone `requestConfirm({...})` che ritorna una `Promise<boolean>` e dispatcha un `CustomEvent` con la funzione `resolve` stessa dentro il payload; `ConfirmModalHost` (montato una volta in `App.jsx`) ascolta l'evento, mostra il modale, e risolve la promise in base al bottone premuto. Così qualunque componente può fare `const ok =`

`await requestConfirm(...)` senza dover gestire stato locale del modale. Aggiunta anche una conferma per l'eliminazione dei commenti, che prima non ne aveva nessuna.

src/utls/confirm.js — l'intero file: una Promise nascosta dentro un CustomEvent

```
export function requestConfirm({ message, title = 'Conferma', confirmLabel = 'Conferma', variant = 'danger' } = {}) {
  return new Promise((resolve) => {
    window.dispatchEvent(
      new CustomEvent('confirm:request', { detail: { message, title, confirmLabel, variant, resolve } }),
    )
  })
}
// uso in un componente qualsiasi, es. PostCard.jsx:
const handleDelete = async () => {
  const confirmed = await requestConfirm({ message: 'Eliminare definitivamente questo post?' })
  if (confirmed) dispatch(deletePost(post.id))
}
```

Il componente `ConfirmModalHost` (montato una volta sola) ascolta l'evento, salva `resolve` nello stato locale, e lo richiama con `true` / `false` a seconda del bottone premuto — questo "sblocca" la `Promise` in attesa nel componente chiamante.

Altri miglioramenti grafici:

- Animazione di ingresso (dissolvenza + leggero scorrimento) per ogni nuovo elemento che compare in una lista: post, commenti, messaggi, risultati di ricerca, notifiche.
- Barra di caricamento sottile in cima alla pagina, gestita da un middleware Redux che conta quante richieste asincrone sono in corso (incrementa su ogni azione `*/pending`, decrementa su `*/fulfilled` o `*/rejected`) invece di essere legata alla sola navigazione tra rotte, che in questa SPA non comporta un vero caricamento a livello di router.
- Dissolvenza progressiva ("blur-up" semplificato) per avatar e immagini dei post: l'elemento `<img>` resta trasparente finché l'evento `onLoad` non conferma che l'immagine è effettivamente pronta, invece di comparire di scatto.
- Pagina 404 riprogettata con lo stesso linguaggio visivo di `EmptyState` (icona in cerchio, dissolvenza in ingresso) al posto del testo semplice di prima.
- Scrollbar personalizzata (sottile, colore coerente con il tema chiaro/scuro) al posto di quella nativa del browser.

Nota di verifica

Durante lo script di test per la presenza online, un primo controllo ("mario riceve `presence:list` alla connessione") è risultato negativo per un motivo *del test*, non del server: il listener veniva registrato dopo l'evento `connect`, quindi in una finestra di tempo in cui il server aveva già inviato l'evento. Corretto registrando il listener prima della connessione; rieseguito, tutti i controlli sono passati.

11 Quattro nuove funzionalità + tre miglioramenti grafici

A differenza dei passaggi precedenti, questo blocco non ha richiesto *nessuna* modifica al backend: tutte le funzionalità riusano endpoint REST già esistenti (`/posts` , `/messages` , `/follows`), quindi non è stato necessario riavviare `npm run dev:all` per vederle attive.

Feed filtrato "Chi segui"

Nuovo toggle in `FeedPage` ("Tutti i post" / "Chi segui"). Sfrutta una caratteristica di json-server non ancora usata prima in questo progetto per i post: passare lo stesso parametro di query più volte (`?userId=3&userId=7&userId=12`) viene interpretato come filtro OR, esattamente come già si faceva per recuperare i "mi piace" di più post in una volta sola. Nuovo thunk `fetchFollowingFeed` con uno stato di paginazione indipendente (`state.posts.followingFeed`), per non interferire con il feed principale. Se l'utente non segue ancora nessuno, non viene fatta nessuna richiesta (altrimenti, senza filtro `userId`, si otterrebbero *tutti* i post): viene mostrato direttamente uno stato vuoto dedicato.

src/components/posts/FollowingFeedList.jsx — la guardia sulla lista vuota

```
const followingIds = useSelector(selectFollowingIds(currentUser.id))

useEffect(() => {
  if (followingIds.length === 0) return // niente da filtrare = niente richiesta
  dispatch(fetchFollowingFeed({ page: 1, limit, followingIds }))
}, [dispatch, limit, followingIds.join(',')])

if (followingIds.length === 0) {
  return <EmptyState icon="bi-person-heart" title="Non segui ancora nessuno" />
}
```

Recibi di lettura visuali

Il campo `read` dei messaggi esisteva già in `db.json` e veniva già aggiornato in tempo reale, ma non era mai stato mostrato nell'interfaccia. Aggiunta un'icona di spunta accanto all'orario di ogni messaggio proprio: singola spunta (`bi-check2`), semitrasparente, per "inviato"; doppia spunta piena (`bi-check2-all`), bianca opaca, per "letto" — stessa convenzione visiva di WhatsApp/Telegram (forma dell'icona, non solo colore, per restare leggibile anche in caso di daltonismo).

Badge dei messaggi non letti in navbar

Il pezzo più delicato di questo blocco. Non essendo pratico scaricare in anticipo tutti i messaggi di tutte le conversazioni solo per contare quelli non letti, il conteggio iniziale viene calcolato con una richiesta mirata per conversazione (`GET /messages?conversationId=X& read=false` , poi filtrata lato client per escludere i messaggi che l'utente ha scritto lui stesso) eseguita in parallelo per tutte le conversazioni al login. Da quel momento in poi il conteggio si mantiene *senza altre richieste*: si incrementa di 1 ad ogni `message:new` ricevuto da un altro utente, e si azzera per una conversazione non appena viene aperta (stesso momento in cui i suoi messaggi vengono comunque segnati come letti). Per farlo funzionare ovunque nell'app (non solo nella pagina Messaggi), l'hook `useConversationSocket` — prima montato solo lì — è stato spostato in `App.jsx`, attivo per l'intera sessione.

src/hooks/useConversationSocket.js — incremento condizionale sull'evento socket

```
function handleNewMessage(message) {
  dispatch(messageReceived(message))
  if (message.userId !== currentUser?.id) { // non e' un messaggio mio
    dispatch(unreadCountIncremented(message.conversationId))
  }
}
// in ConversationView.jsx, all'apertura della conversazione:
useEffect(() => {
  if (conversation) {
    dispatch(fetchMessages({ conversationId, page: 1 }))
    dispatch(conversationMarkedRead(conversationId)) // azzera subito il badge
  }
}, [dispatch, conversationId, conversation])
```

Liste di follower e seguiti

I contatori nel profilo (già presenti dal passaggio 9) sono diventati due pulsanti che aprono un modale con l'elenco effettivo delle persone, riusando lo stesso componente `UserResultItem` già scritto per i risultati di ricerca (stesso avatar, stesso pallino online, stesso link al profilo) invece di crearne uno nuovo.

Popover di mini-profilo al passaggio del mouse

Nuovo componente generico `UserHoverCard`: avvolge qualunque contenuto (avatar, nome) e mostra, al passaggio del mouse o al focus da tastiera, un popover con avatar più grande, bio, numero di follower e un link "Vedi profilo" — senza dover ripetere questa logica in ogni punto dell'app che mostra un utente. Applicato all'autore di un post e all'autore di un commento.

Lightbox sulle immagini dei post

Cliccando su un'immagine allegata a un post si apre un modale a schermo quasi intero con l'immagine ingrandita (sfondo scuro, clic per richiudere) invece di vederla solo nelle dimensioni fisse della card.

Copertina di profilo personalizzabile

Stesso meccanismo già usato per l'avatar (upload di file, `FileReader.readAsDataURL`, limite 2MB lato client, nessun object storage) esteso a un nuovo campo `coverUrl` sull'utente — json-server non ha uno schema fisso, quindi non serve nessuna migrazione: il campo viene semplicemente salvato/letto se presente. Se l'utente non ne carica una, resta il dégradé di default usato finora.

6. File principali toccati in questa sessione

Area	File nuovi	File modificati (principali)
Backend realtime	server/realtime.js	server/server.js, server/db.json, server/db.seed.json, server/routes.json
Frontend realtime/notifiche	src/socket.js, src/hooks/useConversationSocket.js, src/features/notifications/*, src/components/notifications/*	src/App.jsx, src/app/store.js, src/features/messages/messagesSlice.js, src/components/messages/*
Ricerca & paginazione	src/features/search/*, src/pages/SearchPage.jsx, src/components/search/*, src/hooks/useDebounce.js	src/api/usersService.js, src/api/postsService.js, src/features/posts/postsSlice.js
Commenti	—	src/api/commentsService.js, src/features/comments/commentsSlice.js, src/components/comments/*

Estetica (1ª tornata)	src/hooks/useTheme.js, src/utils/toast.js, src/components/common/ToastHost.jsx, src/app/toastMiddleware.js, src/components/common/Skeleton.jsx, src/components/common/RowSkeleton.jsx, src/components/posts/PostCardSkeleton.jsx	src/index.css, index.html, src/components/layout/AppNavbar.jsx, src/components/common/EmptyState.jsx, src/components/posts/PostCard.jsx
Seguire utenti	src/features/follow/followSlice.js	server/realtime.js, server/db.json, server/db.seed.json, server/routes.json, src/api/usersService.js, src/app/store.js, src/components/profile/ProfileCard.jsx, src/pages/ProfilePage.jsx, src/components/notifications/NotificationBell.jsx
Stato online	src/features/presence/presenceSlice.js, src/hooks/usePresenceSocket.js	server/realtime.js, src/App.jsx, src/components/common/Avatar.jsx, src/components/messages/ConversationList.jsx, src/components/search/UserResultItem.jsx
Modal di conferma	src/utils/confirm.js, src/components/common/ConfirmModalHost.jsx	src/App.jsx, src/components/posts/PostCard.jsx, src/components/messages/ConversationView.jsx, src/components/comments/CommentList.jsx
Estetica (2ª tornata)	src/components/common/TopLoadingBar.jsx, src/app/loadingBarMiddleware.js	src/index.css, src/app/store.js, src/pages/NotFoundPage.jsx, src/components/common/Avatar.jsx, src/components/posts/PostCard.jsx, src/components/comments/CommentItem.jsx, src/components/messages/MessageBubble.jsx, src/components/notifications/NotificationBell.jsx
Feed "Chi segui"	src/components/posts/FollowingFeedList.jsx	src/api/postsService.js, src/features/posts/postsSlice.js, src/pages/FeedPage.jsx
Ricevute di lettura	–	src/components/messages/MessageBubble.jsx
Badge messaggi non letti	–	src/api/messagesService.js, src/features/messages/messagesSlice.js, src/hooks/useConversationSocket.js, src/App.jsx, src/pages/MessagesPage.jsx, src/components/messages/ConversationView.jsx, src/components/layout/AppNavbar.jsx
Liste follower/seguiti	src/components/profile/FollowListModal.jsx	src/features/follow/followSlice.js, src/components/profile/ProfileCard.jsx, src/pages/ProfilePage.jsx
Popover mini-profilo	src/components/common/UserHoverCard.jsx	src/components/posts/PostCard.jsx, src/components/comments/CommentItem.jsx
Lightbox immagini post	–	src/components/posts/PostCard.jsx
Copertina profilo	–	src/components/profile/ProfileEditForm.jsx, src/components/profile/ProfileCard.jsx, src/index.css
Documentazione	–	CHECKPOINT.md, README.md

✓ Stato attuale

`npm run lint` e `npm run build` puliti dopo ogni fase. `npm run dev:all` riavviato e riverificato dopo ogni modifica al backend (l'ultimo blocco di funzionalità non ha toccato il backend, quindi non ha richiesto riavvio). Tutte le modifiche sono nella working tree del branch `Lucho`, non ancora commitate: da rivedere e committare quando l'utente lo ritiene opportuno.